

Ecole Nationale d'Ingénieurs de Carthage
Université de Carthage

2ème année Génie Informatique
Programmation des Systèmes Réseaux

SYSTÈME DE MESSAGERIE
DISTRIBUÉE
"QUANTUM TWIN"

Réalisé par :

Maram HADJ ALI

Douaa BEN MARZOUK
Rached ILEHI

Nour BEN TAHER

Encadré par :

Mme Khaoula BEDOUI

Année universitaire : 2025–2026

Table des matières

1	Contexte et enjeux des systèmes distribués modernes	4
2	Fonctionnement général	5
2.1	Données manipulées	5
2.1.1	Données des clients	5
2.1.2	Données des credentials	5
2.1.3	Données des paires	5
2.1.4	Données d'historique	5
2.2	Opérations	6
3	Phase 1 — Architecture TCP initiale	6
3.1	Présentation	6
3.2	Serveur C — server.c	6
3.3	Client terminal — client.c	7
3.4	Protocole de communication	8
3.5	Diagramme de cas d'utilisation	9
3.6	Communication multi-thread	9
4	Synchronisation des jumeaux numériques	10
4.1	Présentation du mécanisme de synchronisation	10
4.2	Données de synchronisation	10
4.2.1	Structure des états clients	10
4.2.2	Registre des paires synchronisées	10
4.2.3	Journal de synchronisation	10
4.3	Cycle de synchronisation réel	11
4.4	Protocole de synchronisation	11
4.5	Gestion de la cohérence et des conflits	12
4.5.1	Protection par mutex	12
4.5.2	Gestion des déconnexions	12
4.6	Diagramme de séquence de synchronisation	13
4.7	Synchronisation dans la Phase 2 (WebSocket)	13
5	Phase 2 — Évolution vers WebSocket, Cloudflare et IA	13
5.1	Limites de la Phase 1 et motivations	13
5.2	Nouvelle architecture en couches	14
5.3	Rôle de chaque composant ajouté	14
5.4	Démarrage de la Phase 2	15
6	Gestion des données	15

7	Visualisation du concept	15
7.1	Intrication quantique et messagerie distribuée	15
7.2	Schéma de messagerie entre jumeaux	16
7.3	Interprétation informatique	16
8	Module d'intelligence artificielle	16
8.1	Présentation	16
8.2	Fonctionnalités	17
8.2.1	Traduction automatique des messages	17
8.2.2	Détection d'anomalies de session	17
8.3	Architecture du module IA	17
9	Conclusion	18
10	Sources	19

Table des figures

1	Architecture initiale TCP — Phase 1	6
2	Diagramme de cas d'utilisation — Phase 1 (LIST réservé à l'admin)	9
3	Routage d'un message entre jumeaux avec gestion multi-thread	9
4	Cycle d'appariement et de propagation d'état entre jumeaux	11
5	Diagramme de séquence — appariement, propagation d'état et déconnexion	13
6	Architecture en couches — Phase 2	14
7	Messagerie bidirectionnelle entre jumeaux via le serveur central	16
8	Architecture du module IA — Groq / LLaMA 3	17

Résumé

Ce projet consiste à concevoir un système de messagerie distribuée entre appareils distants, inspiré du concept d'intrication quantique.

Le système a été développé en deux phases successives. Dans un premier temps, un serveur central en C et un client terminal ont été réalisés en utilisant le protocole TCP avec une gestion multi-thread. Dans un second temps, le système a évolué vers une architecture plus ouverte : un proxy WebSocket, un tunnel Cloudflare sécurisé et une interface mobile moderne ont été ajoutés pour permettre l'accès depuis n'importe quel smartphone sur n'importe quel réseau.

Deux clients appariés, appelés *jumeaux numériques*, peuvent s'échanger des messages en temps réel via le serveur central. Un module d'intelligence artificielle (Groq / LLaMA 3) intégré à l'interface mobile permet la traduction automatique des messages et la détection d'anomalies de session.

1 Contexte et enjeux des systèmes distribués modernes

Selon une étude de Grand View Research, le marché mondial des logiciels de collaboration en équipe était estimé à **36 114,2 millions de dollars** en 2024 et devrait atteindre **57 403,8 millions de dollars** d'ici 2030, avec un taux de croissance annuel moyen de **7,4 %**.

Cette croissance s'explique par l'augmentation du travail à distance ainsi que par le besoin croissant de messagerie et de communication en temps réel entre les utilisateurs. Ces tendances justifient pleinement l'intérêt du projet *Quantum Twin*.



Scannez le code QR pour accéder à l'étude complète

2 Fonctionnement général

Le système repose sur la création de paires de clients. Une fois deux clients appariés (jumeaux), ils peuvent s'échanger des messages en temps réel. Le serveur central reçoit chaque message et le relaie immédiatement vers le destinataire.

2.1 Données manipulées

2.1.1 Données des clients

Chaque client est défini par un identifiant unique attribué à la connexion, un état booléen (ON/OFF) et un statut de connexion (`connecte`/`deconnecte`).

Fichier `clients.txt` :

```
ID Etat Statut
1  ON   connecte
2  ON   connecte
```

2.1.2 Données des credentials

L'authentification repose sur le fichier `credentials.txt` qui associe chaque utilisateur à un mot de passe et un rôle (`admin` ou `user`) :

```
admin secret123 admin
alice pass123   user
bob   pass456   user
```

L'utilisateur `admin` avec le mot de passe `secret123` se voit attribuer le rôle **admin**, ce qui lui donne accès exclusif à la commande `LIST`.

2.1.3 Données des paires

Fichier `pairs.txt` :

```
Client1 Client2
1        2
```

2.1.4 Données d'historique

Fichier `histo.txt` :

Timestamp	ID	Action	Valeur	Résultat
2025-06-01 10:00:01	1	CONNECT	admin	succes
2025-06-01 10:00:05	2	CONNECT	alice	succes
2025-06-01 10:00:10	1	SYNC	2	succes
2025-06-01 10:00:20	1	MSG	"Salut alice!"	envoye
2025-06-01 10:00:25	2	MSG	"Salut admin!"	envoye

2.2 Opérations

Côté client :

- Connexion et authentification au serveur (**CONNECT**)
- Demande d'appariement avec un autre client (**SYNC**)
- Modification de l'état local et propagation au jumeau (**STATE**)
- Envoi de messages à n'importe quel client connecté (**MSG**)
- Test de connectivité (**PING**)
- Consultation de la liste des clients (**LIST**, **admin uniquement**)
- Déconnexion propre (**QUIT**)

Côté serveur :

- Gestion des connexions multi-thread (un thread par client)
- Authentification via `credentials.txt` avec attribution de rôle
- Routage des messages entre clients (`handle_msg()`)
- Propagation d'état vers le jumeau lors d'un **STATE**
- Persistance dans les fichiers texte avec protection par mutex
- Historisation de toutes les actions
- Watchdog : redémarrage automatique en cas de crash

3 Phase 1 — Architecture TCP initiale

3.1 Présentation

La première phase du projet repose entièrement sur le protocole **TCP**. Un serveur central écrit en C gère les connexions entrantes et un client terminal interactif permet de s'y connecter depuis un PC ou un appareil Android via Termux. Cette phase constitue le cœur fonctionnel du système : authentification, appariement et messagerie entre jumeaux sont entièrement opérationnels à ce stade.

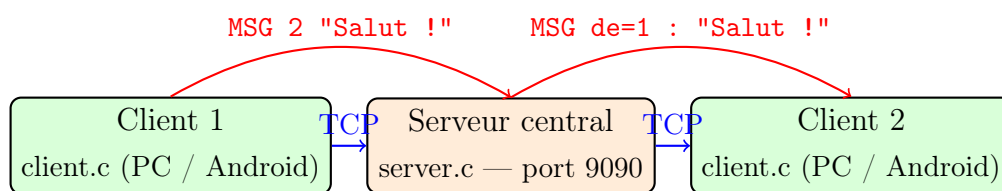


FIGURE 1 – Architecture initiale TCP — Phase 1

3.2 Serveur C — `server.c`

Le serveur est implémenté en C avec les caractéristiques suivantes :

- **Multi-thread** : chaque client accepté génère un thread dédié via `pthread_create()`, permettant de gérer plusieurs connexions simultanées sans blocage.

- **Authentication** : lecture de `credentials.txt`, attribution d'un identifiant unique et d'un rôle (`admin` / `user`). La structure `ThreadArg` conserve le drapeau `is_admin` pour toute la durée de la session.
- **Messagerie** : la fonction `handle_msg()` route les messages entre clients via leur socket active.
- **Propagation d'état** : la fonction `handle_state()` met à jour l'état du client émetteur *et* notifie automatiquement son jumeau via `UPDATE [valeur] from=[id]`.
- **Persistance** : toutes les actions sont enregistrées dans les fichiers texte, protégés par des mutex (`pthread_mutex_t`).
- **Watchdog** : le serveur est lancé via `fork()`. Le processus père surveille le fils et le redémarre après 2 secondes en cas de crash, avec journalisation dans `watchdog.log`.
- **Mode UDP** (bonus) : activé par l'argument `--udp`, illustrant la différence avec TCP.

3.3 Client terminal — `client.c`

Le client C offre une interface interactive en ligne de commande. Un thread de réception tourne en arrière-plan pour afficher les messages entrants sans bloquer la saisie. Il est compatible PC (Linux/WSL) et Android (Termux). Un client Python équivalent (`client.py`) est également fourni.

3.4 Protocole de communication

Commande	Direction	Effet
CONNECT user pass	Client → Serveur	Authentification, attribution d'
SYNC [id]	Client → Serveur	Appariement avec un jumeau
STATE [ON OFF]	Client → Serveur	Modification de l'état + propag
MSG [id] [texte]	Client → Serveur	Envoi d'un message à n'importe
PING	Client → Serveur	Test de connectivité
LIST	Client → Serveur	Liste clients (admin uniquement)
QUIT	Client → Serveur	Déconnexion propre
OK CONNECTED id=[id] role=[role]	Serveur → Client	Confirmation + identifiant + r
MSG de=[id] : [texte]	Serveur → Client	Message reçu
UPDATE [ON OFF] from=[id]	Serveur → Client	Propagation de l'état du jumeau
PONG	Serveur → Client	Réponse au PING
OK [message]	Serveur → Client	Confirmation
CLIENTS\n id=[id] state=[s] status=[st]	Serveur → Client	Réponse à LIST
ERROR [message]	Serveur → Client	Signalement d'une erreur

TABLE 1 – Protocole de communication — Phase 1

3.5 Diagramme de cas d'utilisation

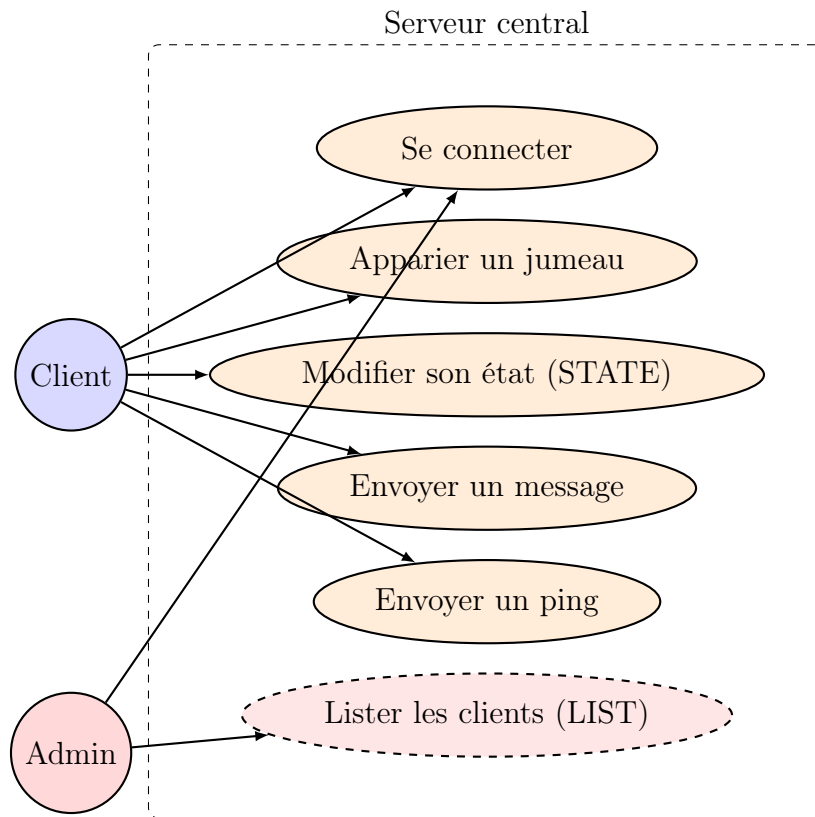


FIGURE 2 – Diagramme de cas d'utilisation — Phase 1 (LIST réservé à l'admin)

3.6 Communication multi-thread

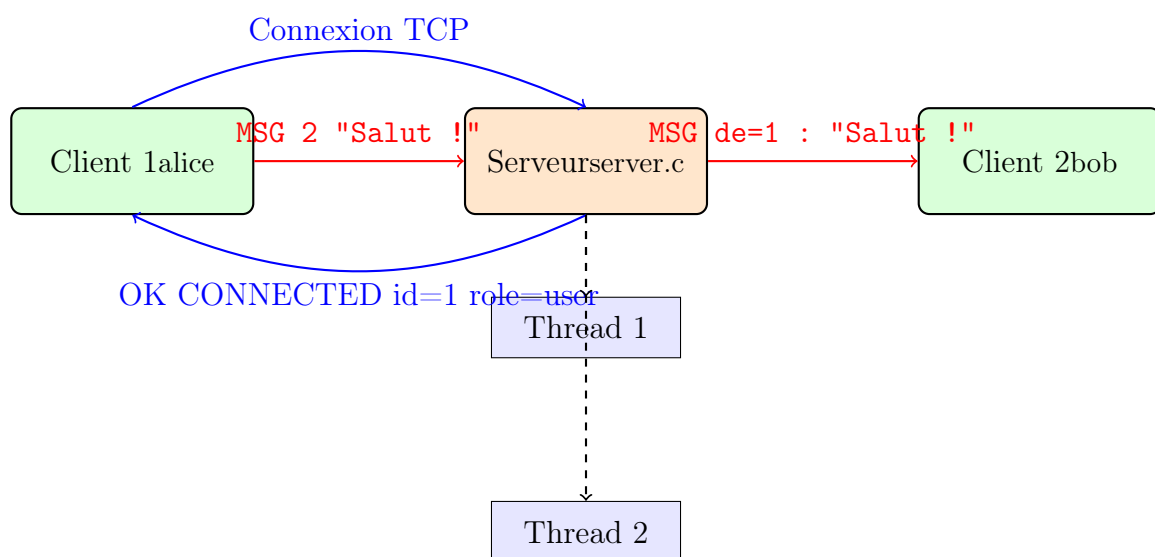


FIGURE 3 – Routage d'un message entre jumeaux avec gestion multi-thread

4 Synchronisation des jumeaux numériques

4.1 Présentation du mécanisme de synchronisation

La synchronisation est le cœur du concept *Quantum Twin*. Elle désigne le processus par lequel deux clients appariés — les jumeaux numériques — maintiennent un état cohérent et partagé en temps réel.

Dans l'implémentation réelle, la synchronisation repose sur deux mécanismes :

- **L'appariement** (SYNC) : établissement du lien entre deux clients, enregistré dans `pairs.txt`.
- **La propagation d'état** (STATE / UPDATE) : toute modification d'état via STATE ON/OFF est immédiatement propagée au jumeau sous forme d'un message UPDATE [valeur] from=[id], et l'état du jumeau est également mis à jour dans `clients.txt`.

Il n'existe pas de commande UNSYNC dans l'implémentation actuelle : le lien entre jumeaux persiste jusqu'à la déconnexion d'un des deux clients, ce qui déclenche la mise à jour du statut dans `clients.txt`.

4.2 Données de synchronisation

4.2.1 Structure des états clients

Chaque client possède un état interne composé d'un identifiant, d'une valeur ON/OFF et d'un statut de connexion. Cet état est stocké dans `clients.txt` et mis à jour en temps réel à chaque modification via STATE ou à la déconnexion.

```
ID Etat Statut
1  ON  connecte
2  OFF connecte
```

4.2.2 Registre des paires synchronisées

Le fichier `pairs.txt` référence toutes les paires actives. Lorsqu'un client envoie SYNC [id_cible], la fonction `add_pair()` vérifie d'abord via `find_twin()` que le client n'est pas déjà apparié. Si ce n'est pas le cas, elle inscrit la paire en mode ajout ("a").

```
Client1 Client2
1       2
```

4.2.3 Journal de synchronisation

Chaque opération est tracée dans `histo.txt` :

Timestamp	ID	Action	Valeur	Résultat
2025-06-01 10:00:10	1	SYNC	2	succes
2025-06-01 10:00:15	1	STATE	ON	succes
2025-06-01 10:00:16	2	UPDATE	ON	recu

4.3 Cycle de synchronisation réel

Le cycle complet tel qu'implémenté dans `server.c` :

1. Le Client A envoie SYNC [id_B] au serveur.
2. Le serveur appelle `handle_sync()` : il vérifie que B existe en mémoire (`get_client()`), puis que A n'est pas déjà apparié (`find_twin()`).
3. Si tout est valide, `add_pair(A, B)` inscrit la paire dans `pairs.txt`.
4. Le serveur confirme à A : OK SYNC paire(A,B) établie.
5. Plus tard, A envoie STATE ON : `handle_state()` met à jour l'état de A dans la table en mémoire.
6. Le serveur recherche le jumeau via `find_twin(A)` et envoie à B : UPDATE ON from=A.
7. L'état de B est *également* mis à jour en mémoire et dans `clients.txt`.
8. Les deux opérations sont tracées dans `histo.txt`.

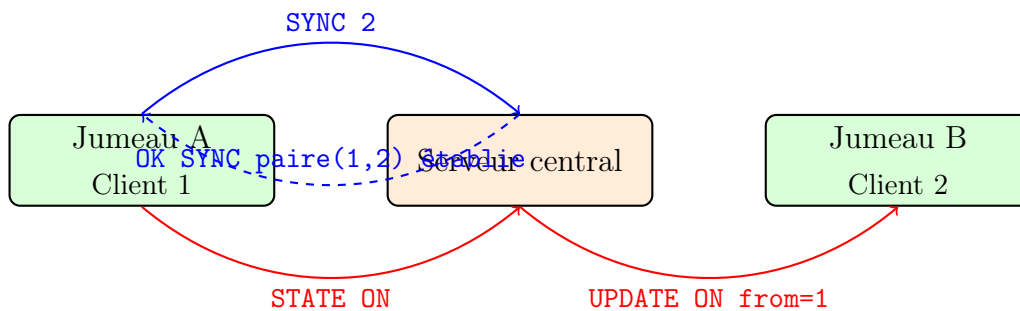


FIGURE 4 – Cycle d'appariement et de propagation d'état entre jumeaux

4.4 Protocole de synchronisation

Les commandes spécifiques à la synchronisation dans l'implémentation :

Commande	Direction	Effet
SYNC [id]	Client → Serveur	Appariement avec un jumeau (si non
STATE [ON OFF]	Client → Serveur	Modification d'état + propagation au
OK SYNC paire(A,B) établie	Serveur → Client	Confirmation de l'appariement
UPDATE [ON OFF] from=[id]	Serveur → Client	Propagation de l'état du jumeau
OK STATE=[val] twin=[id] notifié	Serveur → Client	Confirmation d'état avec jumeau info
OK STATE=[val] twin=[id] hors-ligne	Serveur → Client	Jumeau introuvable lors de la propag

TABLE 2 – Protocole de synchronisation — commandes réellement implémentées

Note : La commande UNSYNC n'est pas implémentée dans cette version. La dissolution du lien est implicite lors de la déconnexion.

4.5 Gestion de la cohérence et des conflits

4.5.1 Protection par mutex

L'accès concurrent aux fichiers de données est protégé par quatre mutex distincts :

- `mtx_clients` : protège les lectures/écritures dans `clients.txt`.
- `mtx_pairs` : protège le registre des paires dans `pairs.txt`.
- `mtx_histo` : protège le journal dans `histo.txt`.
- `mtx_mem` : protège la table clients en mémoire (`g_clients[]`).

Chaque thread acquiert le mutex correspondant avant toute opération, garantissant ainsi la cohérence même en cas d'accès simultanés.

4.5.2 Gestion des déconnexions

Lorsqu'un client se déconnecte (fin de `recv()`, ou commande QUIT), le serveur :

1. Appelle `disconnect_client(id)` : marque le statut `deconnecte` et met `sockfd = -1`.
2. Enregistre l'événement DISCONNECT dans `histo.txt`.
3. Ferme le socket et libère la structure `ThreadArg`.

La prochaine tentative de propagation d'état vers ce client échouera silencieusement (`send_to_client()` retourne `-1` si `sockfd < 0`).

4.6 Diagramme de séquence de synchronisation

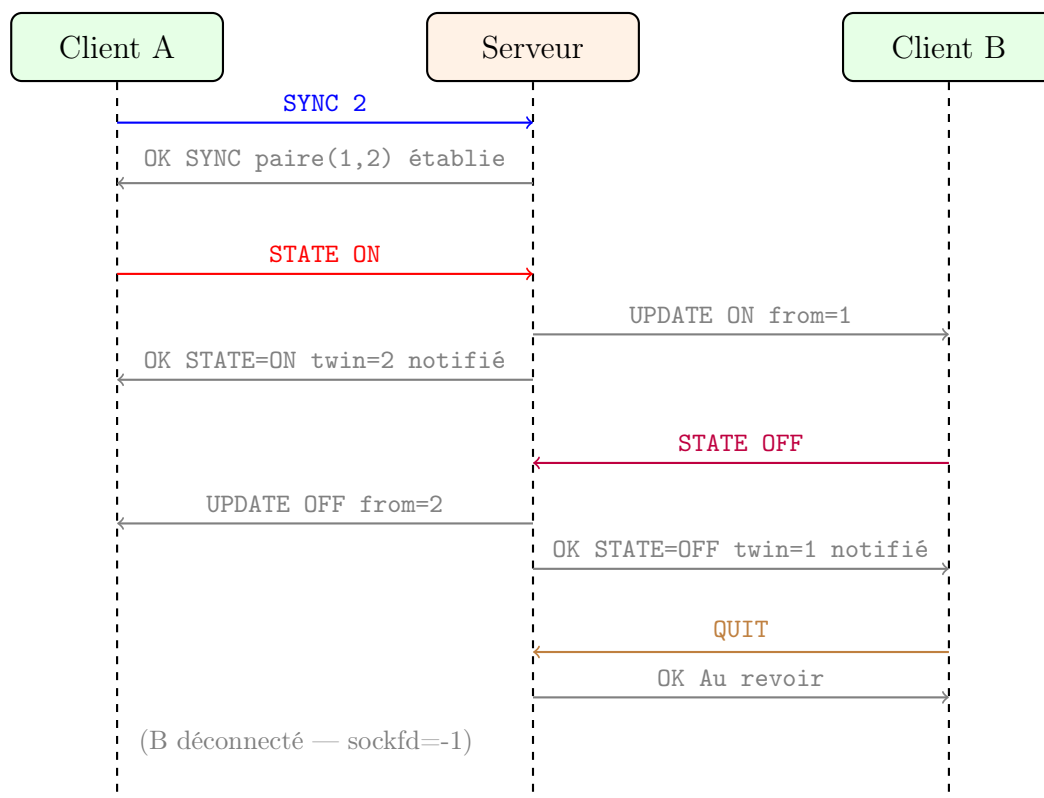


FIGURE 5 – Diagramme de séquence — appariement, propagation d'état et déconnexion

4.7 Synchronisation dans la Phase 2 (WebSocket)

Dans la Phase 2, le mécanisme de synchronisation reste identique côté serveur C. Le proxy `bridge.py` traduit de manière transparente les trames WebSocket en messages TCP bruts, de sorte que les commandes `SYNC`, `STATE` et `UPDATE` transitent sans modification à travers la pile WebSocket-TCP.

L'interface mobile `client.html` affiche en temps réel l'état du jumeau dans l'onglet **SYNC** : connexion, état courant, et historique des synchronisations. Le module IA peut également analyser ces événements pour détecter des comportements anormaux.

5 Phase 2 — Évolution vers WebSocket, Cloudflare et IA

5.1 Limites de la Phase 1 et motivations

La Phase 1 fonctionnait parfaitement en réseau local. Cependant, plusieurs obstacles empêchaient l'accès depuis un smartphone sur n'importe quel réseau :

Problème	Cause	Solution adoptée
Navigateur refuse TCP brut	Sécurité du navigateur	bridge.py (WS → TCP)
PC sur WSL	IP interne inaccessible	Tunnel Cloudflare
ngrok bloque WebSocket	Limitation plan gratuit	Remplacé par cloudflared
IP WSL change au redémarrage	WSL dynamique	portproxy Windows

TABLE 3 – Problèmes rencontrés et solutions adoptées en Phase 2

5.2 Nouvelle architecture en couches

La Phase 2 ajoute trois composants entre le smartphone et le serveur C :

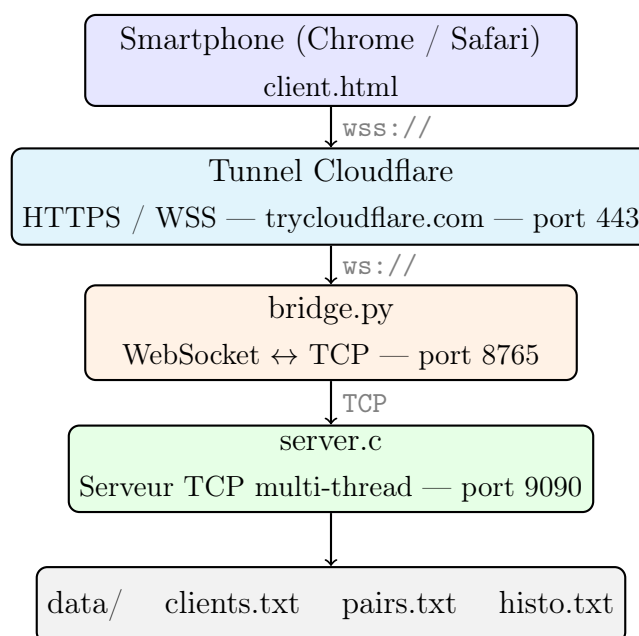


FIGURE 6 – Architecture en couches — Phase 2

5.3 Rôle de chaque composant ajouté

bridge.py est un pont asynchrone écrit en Python (`asyncio` + `websockets`). Il écoute sur le port 8765 en WebSocket et relaie chaque trame vers le serveur C en TCP brut, et inversement. Le serveur C reste ainsi inchangé tout en devenant accessible depuis un navigateur.

cloudflared expose les ports locaux WSL sur Internet via des URLs publiques sécurisées sans configuration de routeur :

- Port 8765 (bridge) → URL `wss://XXXX.trycloudflare.com`
- Port 8080 (client.html) → URL `https://YYYY.trycloudflare.com`

client.html est l'interface mobile accessible depuis n'importe quel smartphone. Elle est organisée en quatre onglets :

- **TERM** — terminal de commandes brutes
- **MSG** — interface de messagerie avec historique, liste des clients connectés (affichage du format `id=X state=Y status=Z`) et traduction IA intégrée
- **SYNC** — appariement avec un jumeau
- **AI** — module d'intelligence artificielle

5.4 Démarrage de la Phase 2

Terminal	Commande	Rôle
1	<code>./server 9090</code>	Serveur C TCP
2	<code>python3 bridge.py</code>	Pont WebSocket ↔ TCP
3	<code>python3 -m http.server 8080</code>	Serveur HTTP pour client.html
4	<code>./cloudflared tunnel --url localhost:8765</code>	Tunnel WSS public
5	<code>./cloudflared tunnel --url localhost:8080</code>	Tunnel HTTPS public

TABLE 4 – Les 5 terminaux WSL à lancer à chaque démarrage

6 Gestion des données

Les données sont stockées dans des fichiers texte pour assurer la persistance entre les sessions. Trois fichiers principaux sont utilisés : `clients.txt`, `pairs.txt` et `histo.txt`, auxquels s'ajoute `credentials.txt` pour l'authentification.

Un mécanisme de verrouillage par mutex (`pthread_mutex_t`) est appliqué sur chaque ressource partagée. Quatre mutex distincts sont définis : `mtx_clients`, `mtx_pairs`, `mtx_histo` et `mtx_mem`.

La table `g_clients[]` maintenue en mémoire évite de relire les fichiers à chaque opération : `get_client()`, `register_client()`, `update_state()` et `disconnect_client()` opèrent directement sur cette table, avec sauvegarde différée dans `clients.txt` via `save_clients()`.

7 Visualisation du concept

7.1 Intrication quantique et messagerie distribuée

Le concept de ce projet s'inspire du phénomène d'intrication quantique, selon lequel deux particules partagent un lien commun indépendamment de la distance qui les sépare. Tout événement affectant l'une se répercute instantanément sur l'autre.

Dans notre système, cette idée se traduit par une **messagerie entre jumeaux numériques** : deux clients appariés sont liés par un canal de communication direct. Tout message envoyé par l'un est immédiatement transmis à l'autre via le serveur central.

Il est important de noter que ce lien repose sur des mécanismes informatiques (TCP, WebSocket, threads) et non sur des phénomènes physiques quantiques réels.

7.2 Schéma de messagerie entre jumeaux

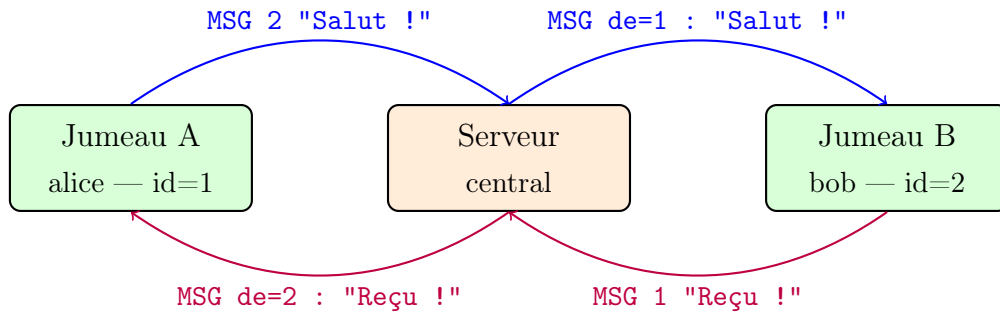


FIGURE 7 – Messagerie bidirectionnelle entre jumeaux via le serveur central

7.3 Interprétation informatique

Lorsqu'un client envoie `MSG [id_cible] [texte]`, la fonction `handle_msg()` du serveur effectue les opérations suivantes :

1. Vérification que le destinataire est enregistré (`get_client()`).
2. Vérification que ce n'est pas un auto-envoi.
3. Récupération de sa socket active (`sockfd`).
4. Construction du message `MSG de=[id] : [texte]`.
5. Envoi immédiat via `send_to_client()`.
6. Enregistrement dans `histo.txt`.

Si le destinataire est hors ligne (`sockfd < 0`), le serveur répond `ERROR Destinataire hors-ligne` à l'expéditeur. La commande `MSG` fonctionne avec n'importe quel client connecté, sans nécessiter d'appariement préalable.

8 Module d'intelligence artificielle

8.1 Présentation

L'interface mobile *client.html* intègre un module d'intelligence artificielle accessible depuis l'onglet **AI**. Ce module utilise l'API **Groq** avec le modèle **LLaMA 3.1-8B-Instant**, un grand modèle de langage (LLM) open-source fonctionnant avec une latence très faible grâce à l'infrastructure LPU de Groq.

8.2 Fonctionnalités

8.2.1 Traduction automatique des messages

Lorsqu'un jumeau reçoit un message dans une langue étrangère, il peut le traduire instantanément en cliquant sur le bouton dédié. Cinq langues cibles sont disponibles : français, anglais, arabe, espagnol et allemand.

La traduction est également disponible **en temps réel lors de la saisie** : avant d'envoyer un message, le bouton dans le compositeur active un panneau de traduction qui suggère automatiquement une traduction au fil de la frappe, avec un délai de 900 ms.

8.2.2 Détection d'anomalies de session

Le module analyse le journal de session (connexions, messages, erreurs, déconnexions) et génère un rapport structuré. Chaque anomalie est classifiée selon quatre niveaux de sévérité :

Niveau	Indicateur	Exemple
critical	CRITIQUE	Déconnexions répétées, erreurs d'authentification
warning	AVERT.	Volume élevé de messages, erreurs isolées
info	INFO	Informations de session normales
ok	OK	Session saine, aucune anomalie

TABLE 5 – Niveaux de sévérité des anomalies IA

La surveillance peut être activée en mode automatique : l'IA analyse la session toutes les 60 secondes et déclenche une analyse immédiate à chaque nouveau message reçu.

8.3 Architecture du module IA

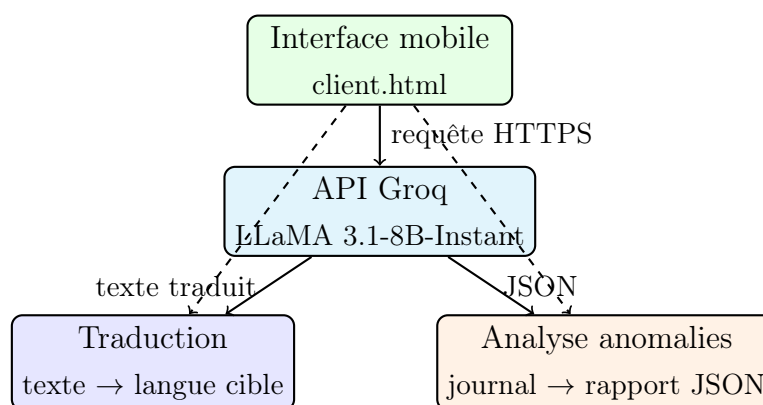


FIGURE 8 – Architecture du module IA — Groq / LLaMA 3

9 Conclusion

Ce projet a suivi une démarche d'évolution progressive en deux phases : une première phase TCP solide servant de socle, puis une seconde phase ouvrant le système au monde via WebSocket, Cloudflare et une interface mobile enrichie d'intelligence artificielle.

Les principaux apports techniques sont :

- Un serveur C multi-thread robuste avec authentification par rôles, watchdog de redémarrage automatique et mode UDP bonus.
- Un mécanisme de synchronisation fiable entre jumeaux numériques, avec propagation d'état bidirectionnelle en temps réel (**STATE** → **UPDATE**), protection par quatre mutex et persistance fichier.
- Un protocole de messagerie simple et extensible (**MSG**) permettant la communication en temps réel entre n'importe quels clients connectés.
- Une gestion des rôles : l'utilisateur **admin** (mot de passe **secret123**) dispose d'un accès exclusif à la commande **LIST** listant tous les clients connectés.
- Une architecture multi-couches (TCP → WebSocket → HTTPS) permettant l'accès depuis un smartphone sur n'importe quel réseau.
- Un module IA intégré offrant traduction en temps réel et surveillance intelligente de la session.

Ce projet ouvre la voie à des extensions futures telles que le chiffrement de bout en bout des messages, la persistance de l'historique des conversations, l'implémentation d'une commande **UNSYNC**, ou l'intégration de modèles IA plus avancés pour l'analyse comportementale des utilisateurs.

10 Sources

- Grand View Research, *Team Collaboration Software Market* :
<https://www.grandviewresearch.com/industry-analysis/team-collaboration-software-market>
- Groq, *LLaMA 3.1 API Documentation* :
<https://console.groq.com/docs>
- Cloudflare, *Cloudflare Tunnel — Quick Tunnels* :
<https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>